

CODEBASE ANALYSIS REPORT

Ticket Management System

GLPI 11.0.9 — Existing Feature Inventory
and Local Run Readiness Assessment

Local project	C:\Users\krishna.shelar\Desktop\Ticket Management System
Upstream project	GLPI — https://github.com/glpi-project/glpi
Product version	11.0.9 (file version/11.0.9)
Git branch	11.0/bugfixes
HEAD commit	76c38071d3 — "Fix/45527/error message wrongly displayed (#25117)"
Licence	GPL-3.0-or-later
Analysis date	11 August 2026
Analysis scope	Read-only static analysis of the working tree. No application file was created, modified or deleted.
Runtime verification	Not performed — the application was not executed. Feature statements are derived from source code, routing, menu definitions and the database schema.

Purpose. This document answers one question: *what already exists in this folder*. It inventories the working, shipped capabilities of the code as checked out, so that the planned customisation work can be scoped as extension rather than rebuild. Section 4 separately records what is still missing to boot the application on this machine.

Important framing. This repository is not a bespoke ticketing application. It is the complete source of GLPI, a mature open-source IT Service Management and IT Asset Management suite. Ticketing is one of seven top-level modules. Roughly 1,600 PHP classes, 478 Twig templates and 442 database tables are already in place.

Generated from static analysis. Figures are counts taken from the working tree at the commit above.

Contents

1. Executive summary	3
1.1 Codebase at a glance	3
2. Technology stack	4
3. Repository map	6
4. Local environment: current state	8
4.1 Prerequisite checks	8
4.2 The uncommitted change to .webpack.config.js	8
4.3 Remaining steps to a running instance	9
5. Existing feature inventory	10
5.1 Navigation map	10
5.2 Assistance — the service desk core	11
5.3 Self-service portal, service catalogue and form builder	15
5.4 Assets and configuration management	17
5.5 Management — financial and contractual	19
5.6 Tools	19
5.7 Administration	21
5.8 Setup	22
6. Cross-cutting platform capabilities	23
7. Quality assets already in the repository	24
8. Where to extend, when customisation starts	25
Appendix A — Key file references	26
Appendix B — Automatic actions present in the code	27

1. Executive summary

The folder contains a full, unmodified checkout of GLPI 11.0.9 on the 11.0/bugfixes maintenance branch. Everything described in section 5 is shipped functionality of that release: it is implemented in the code present here, wired into the navigation, and backed by tables in the bundled database schema.

Three conclusions matter for the plan of work.

- **The ticketing feature set is already deep.** Incidents and requests, a full ITIL lifecycle, SLA and OLA with escalation, approval workflows, templates, a business-rules engine, satisfaction surveys, recurring tickets, costs, and a unified activity timeline are all present. A custom feature list will most likely map onto configuration and extension points rather than new subsystems.
- **GLPI 11 added a no-code form builder and service catalogue.** `src/Glpi/Form` is the single largest module in the codebase (261 files). Custom intake forms, conditional logic, and routing a submission to a Ticket, Change or Problem are achievable through the administration UI with no PHP at all. This is the first place to look for any requested request-intake customisation.
- **The application is not yet runnable on this machine.** PHP and MySQL/MariaDB were not found, the front-end asset bundle has not been built, and no database configuration exists, so GLPI has not been installed. Section 4 records the exact state of each prerequisite.

1.1 Codebase at a glance

Dimension	Count	Notes
Product version	11.0.9	Maintenance branch 11.0/bugfixes; the 11.0 line is the current major series
Domain classes	623	<code>src/*.php</code> — legacy global namespace (Ticket, Change, Problem, Computer, User, Entity, ...)
Namespaced classes	964	<code>src/Glpi/**</code> — modern code: Form, Api, Controller, Console, Asset, Inventory, ...
Legacy page entry points	315	<code>front/*.php</code> — list and form pages, still the bulk of the UI routing
Symfony controllers	74	<code>src/Glpi/Controller/**</code> — the newer routed layer
AJAX endpoints	119	<code>ajax/*.php</code>
Twig templates	478	<code>templates/**</code>
Database tables	442	<code>install/mysql/glpi-empty.sql</code> (was 391 in GLPI 10, 237 in 0.85)
Translations	67 locales	<code>locales/*.po</code> , gettext, including right-to-left layouts
CLI commands	~70	<code>src/Glpi/Console/**</code>
Automatic actions	~48	cron handler methods across the domain classes
Business-rule engines	34	<code>src/Rule*Collection.php</code>
PHPUnit test files	621	<code>tests/functional</code> , <code>tests/units</code>
Cypress end-to-end specs	39	<code>tests/cypress/e2e</code>

2. Technology stack

Read left to right: this is what has to exist on the host, and what any customisation will be written against.

Layer	Technology	Detail as configured in this checkout
Language	PHP 8.2 or newer, 64-bit	composer.json requires php-64bit. Mandatory extensions: bcmath, ctype, curl, dom, fileinfo, filter, gd, hash, iconv, intl, json, libxml, mbstring, mysqli, openssl, session, simplexml, tokenizer, zlib. Suggested: bz2, exif, ldap, phar, zip, Zend OPcache.
Framework	Symfony 6.4 components	HttpKernel, Routing, DependencyInjection, Console, Cache, Config, Validator, Mailer, CssSelector, DomCrawler. Custom kernel in src/Glpi/Kernel. Not a full-stack Symfony app — a hybrid with the legacy front/ pages served through LegacyFileLoadController.
Templating	Twig	478 templates under templates/: pages/, components/, layout/, central/. Twig is also the engine for notification content templates and external links.
Data access	Custom active-record layer	CommonDBTM and its descendants (CommonDropdown, CommonTreeDropdown, CommonDBRelation, CommonDBChild, CommonITILObject) plus the DBmysqlIterator query builder. No Doctrine.
Database	MariaDB 10.6+ or MySQL 8.0+	InnoDB, utf8mb4, timezone tables recommended. Dev stack pins MariaDB 11.8.
Front-end build	Webpack 5, two configs	.webpack.config.js builds one bundle per file in lib/bundles (22 entries); .vue.webpack.config.js builds the Vue 3 application. Output goes to public/build.
UI libraries	Tabler, jQuery, Vue 3	Tabler SCSS design system, TinyMCE rich text, Monaco code editor, FullCalendar, GridStack dashboards, ECharts charts, Leaflet maps, Cytoscape impact graphs, SortableJS, PhotoSwipe, Flatpickr, Cropper, Masonry, Altcha.
Styling	SCSS compiled server-side and at build	css/ sources plus scssphp/scssphp for runtime palette compilation, enabling custom themes from files/_themes.
Authentication	Eight mechanisms	Local database, LDAP or Active Directory, IMAP/POP mail server, external/SSO variable, CAS, X.509 certificate, API token, persistent cookie. TOTP two-factor on top.
Authorisation server	league/oauth2-server	GLPI acts as an OAuth2 provider for its own API; also an OAuth2 client for Google and Azure identity and for SMTP/IMAP XOAUTH2.
APIs	Three surfaces	Legacy REST API (documented in api-rest.md), High-Level REST API v2.0 to v2.3 with generated OpenAPI and Swagger UI, and a GraphQL endpoint.
Mail	Laminas Mail and Mime	Outbound through GLPIMailer; inbound collection through MailCollector over IMAP/POP.
Export	TCPDF, PhpSpreadsheet, league/csv	Search results export to CSV, PDF portrait, PDF landscape, ODS, XLSX, and a names-only list.
Calendaring	sabre/dav and sabre/vobject	A CalDAV server publishes GLPI planning to external calendar clients. RRULE recurrence via rlanvin/php-rrule.
Caching	Symfony Cache	Filesystem by default; Redis and Memcached supported and configurable from the CLI.
Logging	Monolog	Application logs to files/_log, surfaced in the built-in log viewer.

Layer	Technology	Detail as configured in this checkout
Quality tooling	Static analysis and linting	PHPStan with three baselines, Psalm, Rector, PHP-CS-Fixer, ESLint, Stylelint.
Test tooling	PHPUnit, Cypress, Jest, Playwright	621 PHPUnit files, 39 Cypress specs, Jest for JS and Vue units, a Playwright config.
Dev environment	Docker Compose	PHP 8.4 app on port 8080, MariaDB 11.8, Mailpit on 8025, DBGate on 9000, OpenLDAP seeded from tests/LDAP/ldif.

3. Repository map

Where things live. This is the orientation map for the customisation phase.

Path	Contents
src/*.php	623 domain classes in the global namespace. The ITIL core is here: <code>CommonITILObject</code> (11,671 lines), <code>Ticket</code> (6,475 lines), <code>Change</code> , <code>Problem</code> , plus actors, tasks, followups, solutions, validations, SLA/OLA, templates and every reference dropdown.
src/Glpi/Form	261 files. The GLPI 11 form builder: questions, sections, conditions, destinations, access control, translations, export/import, service catalogue integration.
src/Glpi/Api	83 files. Legacy REST, High-Level REST v2.0–v2.3 with 21 resource controllers, GraphQL, RSQL filter parser, OpenAPI generator.
src/Glpi/Console	74 files, roughly 70 CLI commands: install, update, cache, plugin, marketplace, migration, user administration, diagnostics, LDAP sync, security keys, maintenance.
src/Glpi/Controller	74 Symfony controllers — the newer routed layer (helpdesk, service catalogue, forms, knowledge base, install, status, MFA, illustrations).
src/Glpi/Asset	59 files. Custom asset type definitions: 30 capacities, 11 custom field types, dictionary rules for custom models and types.
src/Glpi/Inventory	45 files. Native agent inventory: 33 asset handlers plus main-asset resolution and request handling.
src/Glpi/Kernel	33 files. Boot, container, environment, listeners.
src/Glpi/System	32 files. Requirement checks, status, diagnostics, log.
src/Glpi/ContentTemplates	30 files. Twig-based notification and template tag engine.
src/Glpi/Dashboard	22 files. Grid editor, 19 widget types, filters, data providers, sharing.
src/Glpi/Search	22 files. Search engine, search options, criteria filter, and 12 output writers (HTML, CSV, PDF, ODS, XLSX, names, map).
src/Glpi/Features	22 reusable traits: <code>Timeline</code> , <code>Kanban</code> , <code>Clonable</code> , <code>PlanningEvent</code> , <code>Teamwork</code> , <code>Inventoriable</code> , <code>AssignableItem</code> , <code>TreeBrowse</code> , <code>DCBreadcrumb</code> , <code>State</code> .
src/Glpi/Stat	16 files. Statistics data providers for global, per-item and per-location views.
front/	315 legacy entry points, plus subfolders <code>asset/</code> , <code>form/</code> , <code>dropdown/</code> , <code>contenttemplates/</code> , <code>log/</code> .
ajax/	119 XHR endpoints backing in-page interactions.
templates/	478 Twig templates.
install/	Installer, <code>empty_data.php</code> seed data, per-version schemas back to 0.85, and <code>migrations/</code> for upgrades.
inc/	Bootstrap and constant definitions for the legacy layer.
config/	Runtime configuration. Currently empty — see section 4.
files/	Runtime data: uploaded documents, logs, cache, sessions, plugins data, themes.
lib/bundles/	22 webpack entry points (<code>base.js</code> , <code>tabler.scss</code> , <code>tinymce</code> , <code>monaco</code> , <code>echarts</code> , <code>leaflet</code> , <code>cytoscape</code> , <code>fullcalendar</code> , <code>gridstack</code> , and others).
public/	Web document root: <code>index.php</code> front controller, built assets, images, sounds.
plugins/ and marketplace/	Drop-in extension directories. Empty in this checkout.

Path	Contents
tests/	PHPUnit suites, Cypress e2e, fixtures, LDIF data for LDAP, email fixtures.
tools/	Maintenance and build scripts, including npm patches applied post-install.
bin/console	Symfony console entry point for all CLI operations.

4. Local environment: current state

Recorded so the run attempt is not a guessing exercise. Dependencies are installed; the runtime is not yet in place.

4.1 Prerequisite checks

Check	State	Evidence and consequence
Composer dependencies	PRESENT	vendor/autoload.php exists and vendor/ is populated. .composer.hash is present, so the install completed.
npm dependencies	PRESENT	796 entries in node_modules/; .package.hash written. package-lock.json tracked.
Node.js	PRESENT — v24.18.0	Sufficient for the webpack build.
PHP CLI	NOT FOUND	php is not on PATH and was not found at the usual Windows locations (XAMPP, Laragon, C:\php, Program Files). Without it, neither bin/console nor the web application can run. Blocking.
MySQL / MariaDB	NOT FOUND	No mysql, mysqld or mariadb binary on PATH. A reachable server is required before the installer can create the schema. Blocking.
Web server	NOT VERIFIED	No Apache, Nginx or IIS configuration was found in the project. The document root must be public/, not the project root.
Database configuration	ABSENT	config/ is empty — no config_db.php. GLPI has therefore never been installed from this checkout. First browser request will redirect to the installer.
Front-end assets	INCOMPLETE	public/build/ contains only vue/ (app.js and the SFC chunks). The 22 webpack bundles from lib/bundles — including base.js and tabler.scss — are missing, so npm run build:pack has not completed. The UI would render unstyled and largely non-functional. Blocking for usable UI.
Local source change	1 FILE MODIFIED	.webpack.config.js is modified and uncommitted — see 4.2.
files/ write access	NOT VERIFIED	The installer checks write access to files/ subdirectories and config/. Verify at run time.

4.2 The uncommitted change to .webpack.config.js

One tracked file differs from HEAD. It is a build-time workaround, not a feature change, and it is worth understanding before the build is retried.

Upstream enumerates the bundle entry points with `globSync(path.resolve(__dirname, 'lib/bundles') + '/*(*.min).js')`. On Windows that resolves to a backslash path, which glob treats as escape characters, so the pattern matches nothing and webpack produces no bundles. The local edit replaces the glob with `fs.readdirSync` plus a suffix filter, and hands webpack a forward-slash relative path:

```
const bundleDir = path.resolve(__dirname, 'lib/bundles');
const files = fs.readdirSync(bundleDir)
  .filter(f => f.endsWith(ext) && !f.endsWith('.min' + ext));
entries[entry_name] = './' + path.posix.join('lib/bundles', file);
```

Assessment. The intent is correct and the change is confined to build configuration — it cannot affect application behaviour. Because public/build/ still lacks the pack output, the build either was not re-run after the edit or failed for another reason. Keep the edit; re-run the build and read the webpack output. Note that it is a local divergence from upstream and will surface on every rebase of 11.0/bugfixes.

4.3 Remaining steps to a running instance

Stated as findings for the run attempt, in dependency order. Nothing here has been executed.

#	Step	Detail
1	Provide PHP 8.2 or newer	64-bit, with the mandatory extensions listed in section 2. Add it to PATH so <code>bin/console</code> works.
2	Provide MariaDB 10.6+ or MySQL 8.0+	Create an empty database and a user with rights on it. Loading the timezone tables enables GLPI's timezone support.
3	Build the front-end assets	<code>npm run build</code> — runs <code>build:pack</code> , then <code>build:vue</code> , then the illustration translations step (which itself needs PHP). Confirm the 22 bundles appear under <code>public/build/</code> .
4	Install the schema	Either the web installer at <code>/install/install.php</code> , or <code>php bin/console database:install</code> . This writes <code>config/config_db.php</code> and creates the 442 tables.
5	Serve <code>public/</code> as the document root	Everything routes through <code>public/index.php</code> . Pointing the virtual host at the project root exposes source and breaks routing.
6	Enable automatic actions	Schedule <code>php front/cron.php</code> (or run <code>cron</code> in GLPI mode) so escalation, notification dispatch, surveys and recurring tickets fire.

Lower-friction alternative on Windows. The repository ships a complete Docker Compose stack that removes steps 1, 2 and 5 entirely: an app container (PHP 8.4 with Apache) on port 8080, MariaDB 11.8, Mailpit on 8025 for capturing outbound mail, DBGate on 9000 for database inspection, and a seeded OpenLDAP for testing directory authentication. Given that PHP and MySQL are both absent from this machine, this is the shorter path to a working instance.

5. Existing feature inventory

The inventory below follows the application's own navigation, which is defined authoritatively in `Html::getMenuInfos()` at `src/Html.php:1250`. Every entry corresponds to code present in this checkout. Where a feature is new in the 11.0 series it is marked **[11.0]**, taken from `CHANGELOG.md`.

5.1 Navigation map

Menu	Modules reachable from it
Assets	Computer, Monitor, Software, Network equipment, Peripheral, Printer, Cartridge, Consumable, Phone, Rack, Enclosure, PDU, Passive DC equipment, Unmanaged device, Cable, plus custom asset types and component types
Assistance	Ticket, Service catalogue, Problem, Change, Planning, Statistics, Recurring ticket, Recurring change
Management	Software licence, Budget, Supplier, Contact, Contract, Document, Line, Certificate, Datacenter, Cluster, Domain, Appliance, Database
Tools	Project, Reminder, RSS feed, Knowledge base, Reservation, Report, Saved search, Impact analysis
Administration	User, Group, Entity, Rules, Profile, Notification queue, Log viewer, Inventory, Forms
Setup	Asset definitions, Dropdowns, Components, Notifications, Webhooks, Service levels, General setup, Field unicity, Automatic actions, Authentication, OAuth clients, Receivers, External links, Plugins
Plugins	Populated at runtime by installed plugins
My settings	Per-user preferences (hidden entry)

5.2 Assistance — the service desk core

This is the module the project is named after, and it is the most developed part of the codebase. Ticket, Change and Problem all extend CommonITILObject, so most capabilities below are shared across the three record types.

5.2.1 Ticket record and lifecycle

Feature	As implemented
Ticket types	Incident and Request (Ticket::INCIDENT_TYPE, Ticket::DEMAND_TYPE). The default type is configurable per entity.
Statuses	New, Processing (assigned), Processing (planned), Pending, Solved, Closed. The shared ITIL status set also carries Approval, Accepted and Observed, used by Changes and Problems. Allowed transitions are driven by a configurable status matrix.
Priority	Computed from an Urgency x Impact matrix that is editable in Setup (priority_matrix, urgency_mask, impact_mask). Five levels plus Major. A dedicated right controls who may override priority.
Actors	Three roles — Requester, Observer, Assigned to — and each accepts users, groups or suppliers. Anonymous requesters identified only by email address are supported, which is what makes email-created tickets work.
Classification	ITIL category (hierarchical tree, shared across Ticket/Change/Problem), request source, location, entity, business criticality.
Associated items	Assets (Item_Ticket), contracts (Ticket_Contract), knowledge base articles, projects and project tasks.
Activity timeline	A single chronological stream merging followups, tasks, documents, solutions, approvals and notifications. Implemented by the Timeline trait.
Followups	Public or private, rich text, optional followup templates, source tracking, and notification on add.
Tasks	Public or private, with task category, planned start and end, actual duration, assigned technician or group, task templates, and planning integration so tasks appear on calendars.
Solutions	Solution types, reusable solution templates, and a redesigned knowledge-base-backed solution search UI [11.0] . Solutions can be approved or rejected by the requester, and a rejection reopens the ticket.
Pending reasons	A ticket can be parked with a reason, with automatic follow-up bumping and automatic solving after a configured delay (PendingReason, PendingReasonCron).
Costs	Time-based, fixed and material costs per ticket, with cost totals (TicketCost on CommonITILCost).
Satisfaction survey	Internal survey or an external URL, triggered by an automatic action after closure. Configurable maximum score, default score and mandatory comment [11.0] . Answers notify back to the team.
Linking	Any ITIL object can be linked to any other — duplicate of, linked to, parent and child [11.0] , replacing the old ticket-only linking. Massive actions <i>Link ITIL Object</i> and <i>Unlink ITIL Object</i> [11.0] .
Escalation to Problem or Change	A ticket can be promoted to, or attached to, a Problem or a Change (Problem_Ticket, Change_Ticket).
Takeover	The <i>steal</i> and <i>own</i> rights let a technician take a ticket already assigned to someone else, or self-assign.
Concurrency	Optimistic object locking warns when two users open the same ticket (ObjectLock).
Kanban board	Drag-and-drop board view over tickets and other supported types, with per-user saved column state that an authorised user can reset [11.0] .
History	Every field change is journalled to glpi_logs and displayed through the search engine [11.0] .

Feature	As implemented
Housekeeping	Automatic closure of solved tickets after N days, purge of closed tickets after N days, and alerts on tickets that are not solved within a threshold.

5.2.2 Approval (validation) workflow

Renamed from *Validation* to *Approval* throughout the 11.0 UI. Available on Tickets and Changes.

Capability	Detail
Approval requests	Requested from a user, from a group's users, or from a whole group where any member may answer [11.0] .
Approval steps	Requests are grouped into ordered validation steps [11.0] (<code>ValidationStep</code> , <code>ITIL_ValidationStep</code>), enabling multi-stage sign-off.
Templates	<code>ITILValidationTemplate</code> and <code>ITILValidationTemplate_Target</code> predefine who is asked and with what wording.
Substitutes	<code>ValidatorSubstitute</code> delegates approval authority during absence.
Reminders	An automatic action chases outstanding approvals [11.0] .
Constraints	Approvals are refused on Tickets and Changes that are already solved or closed [11.0] .
Status rollup	A global approval status on the parent record drives notifications and the Approval status.

5.2.3 Service levels: SLA and OLA

Capability	Detail
Two agreement types	SLA measures the commitment to the requester; OLA measures internal handling. Both derive from <code>LevelAgreement</code> and both support a time-to-own and a time-to-resolve target.
Working-hours awareness	Targets are computed against a <code>Calendar</code> with working segments and holidays, so out-of-hours time is excluded.
Escalation levels	<code>SlaLevel</code> and <code>OlaLevel</code> fire at configurable offsets before or after the deadline, each with its own criteria and actions — reassign, change priority, notify, add a followup.
Automatic actions	<code>cronSlaTicket</code> and <code>cronOlaTicket</code> evaluate pending escalations; <code>recall</code> and <code>recall_ola</code> notification events send deadline reminders.
Assignment right	A dedicated right controls who may attach a service level to an ITIL object [11.0] .

5.2.4 Templates and business rules

Capability	Detail
ITIL templates	<code>TicketTemplate</code> , <code>ChangeTemplate</code> and <code>ProblemTemplate</code> each declare mandatory, hidden, predefined and read-only fields. Templates apply by entity, category, type and profile, and can now also restrict the selectable statuses [11.0] .
Child templates	Reusable templates for followups (<code>ITILFollowupTemplate</code>), tasks (<code>TaskTemplate</code>), solutions (<code>SolutionTemplate</code>) and approvals.
Business rules	A rules engine runs on create and on update. Rules for tickets have existed for years; rules for Changes and for Problems are new [11.0] . Criteria cover virtually every field plus the requester's profile, group and location; actions assign, reclassify, set urgency or priority, add actors, attach a service level, or send a notification.
Rule tooling	Rules can be reordered, individually tested against a sample (<code>front/rule.test.php</code>), replayed over existing records, and exported or imported as backups.
Category from code	A rule action maps an inbound code to an ITIL category — useful for email and API intake.

5.2.5 Recurring work, Problems, Changes

Module	Detail
Recurring tickets	<code>TicketRecurrent</code> generates tickets on a schedule from a template, with a calendar, a creation lead time, and associated elements [11.0] . Driven by <code>cronRecurrentItems</code> .
Recurring changes	<code>RecurrentChange</code> provides the same mechanism for Changes.
Problem management	Full records with impact, cause and symptom fields, tasks, solutions, costs, templates, business rules [11.0] , supplier and user actors, and links to the tickets and changes they relate to.
Change management	Change records carrying the approval workflow, planning, rollback and checklist content, tasks, costs, templates, business rules [11.0] , and satisfaction surveys [11.0] .

5.2.6 Planning and statistics

Feature	Detail
Planning	Day, week and month calendar views over ticket tasks, change tasks, problem tasks, project tasks, reservations and standalone external events. Per-user, per-group and filtered views.
External events	<code>PlanningExternalEvent</code> with templates, categories, RRULE-based recurrence, and the ability to detach a single occurrence from its series [11.0] .
Reminders	<code>PlanningRecall</code> sends a notification a configurable interval before an event.
Calendar publication	A CalDAV server exposes GLPI planning to Outlook, Thunderbird and mobile clients; ICS export is also available.
Statistics	Four analyses, each available for Ticket, Problem and Change: <i>Global</i> , <i>By ticket/problem/change</i> , <i>By hardware characteristics</i> , and <i>By hardware</i> . Counts and average times to take into account, to resolve and to close, rendered as charts and exportable.
Homepage widgets	The central view shows personal views, planning for the coming period, reminders, RSS items and dashboards.

5.2.7 Ticket notification events

Fourteen ticket-specific events, each with its own template and its own recipient list. Recipients may be resolved dynamically (requester, technician, group manager, supervisor, watcher, supplier) and exclusions are supported **[11.0]**.

New ticket	Approval request answer	Automatic reminders of SLAs
Update of a ticket	Approval reminder	Automatic reminders of OLAs
Ticket solved	Closing of the ticket	Satisfaction survey

Solution rejected	Deletion of a ticket	Satisfaction survey answer
Approval request	Not solved tickets	

5.3 Self-service portal, service catalogue and form builder

The flagship area of GLPI 11 and, at 261 files, the largest single module in the codebase. It is the part most likely to absorb custom requirements without any PHP being written.

5.3.1 Helpdesk home and service catalogue

Feature	Detail
Simplified interface	A separate self-service UI for end users, selected by profile, with its own layout, navigation and rights.
Configurable tiles	The home page is assembled from tiles: a form tile, a GLPI page tile, or an external page tile. Tiles are ordered by drag and drop, configured per entity, and can be copied from the parent entity (<code>src/Glpi/Helpdesk/Tile</code>).
Service catalogue	A browsable, searchable catalogue of request types organised into categories, with pluggable sort strategies and its own search manager (<code>src/Glpi/Form/ServiceCatalog</code>).
FAQ	Knowledge base articles flagged as FAQ are published to the portal, optionally to anonymous visitors.
Translations	The portal's own strings are translatable per entity (<code>HelpdeskTranslation</code>).
Ticket follow-up	End users can see their own tickets, add followups, approve or reject solutions, and answer satisfaction surveys.

5.3.2 Form builder

Capability	Detail
Structure	Forms are composed of sections and blocks, with free-text comment blocks; layout is configurable per question.
Question types	Around 30 built-in types — listed below.
Conditional logic	A condition engine shows or hides questions, sections and destinations depending on prior answers, edited in a dedicated visual editor (<code>src/Glpi/Form/Condition</code>).
Destinations	One submission can create several records. <code>FormDestinationTicket</code> , <code>FormDestinationChange</code> and <code>FormDestinationProblem</code> each expose per-field strategies: take the value from an answer, use a fixed value, or fall back to the default.
Access control	Policies per form: an allow list of users, groups and profiles; a direct-access token for unauthenticated links; and a mandatory-authentication switch (<code>src/Glpi/Form/AccessControl</code>).
Delegation	A user may submit a form on behalf of another (<code>DelegationData</code>).
Translation	Every label and helper text is translatable per form (<code>FormTranslation</code>).
Export and import	Forms serialise to a portable file; import runs as a four-step wizard — upload, preview, resolve missing references, execute.
Tags	Answer values can be injected into destination content through a form tag system.
Answer storage	Submissions persist as <code>AnswerSet</code> records linked to the objects they created, so the original submission stays auditable.
Draft cleanup	<code>cronPurgeDraftForms</code> removes abandoned drafts.
Legacy migration	<code>FormCreatorPluginToCoreCommand</code> migrates forms from the old <code>FormCreator</code> plugin into the core builder.
Bot protection	Altcha proof-of-work CAPTCHA is available on anonymous forms.

5.3.3 Available question types

Short text	Date and time	User device
Long text	Dropdown (custom values)	Requester (actor)
Number	Radio buttons	Observer (actor)
Email	Checkboxes	Assignee (actor)
URL-safe short answer	File upload	Urgency
Date	Item (any GLPI object)	Request type
Time	Item dropdown	ITIL category

Actor, item and selectable types carry their own extra-data configuration classes, so the set is extensible: a new type is a class implementing `QuestionTypeInterface` registered with `QuestionTypesManager`.

5.4 Assets and configuration management

Area	Detail
Native asset types	Computer, Monitor, Software, Network equipment, Peripheral, Printer, Cartridge item, Consumable item, Phone, Rack, Enclosure, PDU, Passive DC equipment, Unmanaged device, Cable.
Custom asset types	AssetDefinition creates entirely new asset types from the UI, each opting into any of 30 capacities — history, documents, notes, links, contracts, infocom, knowledge base, impact, devices, network ports, operating system, software, virtual machines, volumes, antivirus, peripherals, sockets, plugs, domains, appliances, certificates, database instances, remote management, inventoriable, rackable, reservable, project asset, global search. Each definition also supports custom fields in 11 types (string, text, number, boolean, date, date-time, dropdown, URL, raw).
Components	Processor, memory, hard drive, network card, graphic card, sound card, drive, controller, battery, camera, case, firmware, generic, motherboard, PCI, power supply, sensor, SIM card — each with its own models and types, and each attachable to any asset.
Software and licences	Software with versions and categories; licences with types, per-item and per-user assignment, and compliance counting. Dictionary rules normalise software names and versions at import.
Networking	Network ports with typed instantiations (ethernet, wifi, aggregate, alias, dialup, local, fibre channel), port-to-port connections, VLANs, IP addresses, IP networks and netmasks, FQDNs, network names and aliases, wifi networks, SNMP credentials, sockets, connection logs and port metrics.
Operating systems	Operating system with edition, version, service pack, architecture, kernel and kernel version, recorded per asset.
Runtime detail	Antivirus, virtual machines, disks and volumes with used-percentage [11.0] , running processes [11.0] , environment variables [11.0] , remote management identifiers.
Reference data	Status, location tree (with alias and code [11.0] , geolocation and map display), manufacturers, models, types, business criticality, user categories, and dozens more dropdowns — all translatable.
Templates and cloning	Asset templates; template cloning [11.0] ; and creating a template from an existing item [11.0] .
Assignment	Assets carry a user and a group, plus a technician and a group in charge; the group fields now accept multiple groups [11.0] . Dedicated <i>View assigned</i> and <i>Update assigned</i> rights [11.0] .
Attachments	Documents, notes, external links, full change history, and impact analysis per asset.
Reservation	Any asset can be made reservable and then booked through the reservation calendar.
Global search	A cross-type search covers every asset type that opts in.

5.4.1 Datacenter infrastructure (DCIM)

Feature	Detail
Hierarchy	Datacenter, then server room, then rack, then rack item. Locations appear in datacenter breadcrumbs [11.0] .
Room grid	Rooms are laid out on a grid whose size is configurable per room [11.0] , with drag-and-drop placement.
Rack modelling	Units, front and rear faces, orientation, enclosures, blade positions.
Power	PDUs and PDU models attached to racks, plugs and plug-to-PDU mapping.
Cabling	Cables with types and strands, terminated on sockets.
Stencils	Visual front-panel templates for network equipment models (Stencil, NetworkEquipmentModelStencil).

5.4.2 Native inventory

Feature	Detail
Agent intake	GLPI Agent posts inventories over HTTP; 33 asset handlers parse them (src/Glpi/Inventory/Asset). No plugin is required — native since GLPI 10.
Import rules	RuleImportAsset decides whether an inventory updates an existing asset or creates a new one; RuleImportEntity and RuleLocation route it to the right entity and location; RuleDefineItemtype picks the asset type.
Dictionaries	Twenty-plus dictionary rule sets normalise manufacturers, models, types, operating systems, printers and software as data arrives.
Agents	Agent registry with versions and last-contact tracking; stale agents are cleaned by cronCleansAgents.
Quarantine	RefusedEquipment holds inventories the rules rejected; Unmanaged holds devices discovered but not yet classified.
Field protection	Lockedfield pins manually corrected values so the next inventory cannot overwrite them.
Configuration	Per-type import behaviour and component handling in the Inventory configuration screen.

5.5 Management — financial and contractual

Module	Detail
Financial information	Infocom attaches purchase date, order and invoice numbers, supplier, warranty duration and terms, value, depreciation (linear or degressive) and end-of-life to any item, with alerting on warranty expiry — which no longer fires for deleted items [11.0] .
Budgets	Budgets with types, periods and value, consumed against items and contracts. The main tab shows overspend in accounting parentheses [11.0] .
Contracts	Contracts with types, durations, notice periods, renewal, billing periodicity, costs, linked suppliers and linked items; alerts before expiry.
Suppliers and contacts	Suppliers with types, and contacts with types, linked to each other and to items.
Documents	Uploaded files with categories and MIME type control, attachable to any item. Drag-and-drop upload, and direct screen capture or screen recording from the ticket attachment form [11.0] .
Software licences	Licence records, counts, assignment to items and users, and compliance views.
Certificates	Certificates with types, expiry dates and item links.
Lines	Telephony and data lines with operators and types, now assignable directly to an asset without requiring a SIM card [11.0] .
Clusters	Cluster records grouping member assets.
Domains	Domains with types, relations and DNS records with record types; attachable to items.
Appliances	Appliances with types and environments, linked to items and to each other.
Databases	Database and database-instance inventory with categories and types.
Datacenters	Also reachable from this menu — see 5.4.1.

5.6 Tools

Module	Detail
Projects	Projects with states, types, priority, percent done, hierarchical sub-projects, costs, and a rich-text description [11.0] . Project tasks with templates, types, dependencies (ProjectTaskLink), effort and planning. Project and task teams drawn from users, groups, suppliers and contacts. Gantt and Kanban views. Links to tickets, changes and problems, and to project assets.
Knowledge base	Articles with a category tree, revision history, comments, per-language translations, and visibility scoped by entity, profile, group or user. Articles can be published to the FAQ, rated by readers, linked to items, and proposed as the solution to a ticket. The browse tab now runs on the search engine [11.0] .
Reminders	Personal and public reminders with visibility rules, planning integration and translations. The central view separates personal reminders (own, non-public) from public ones [11.0] .
RSS feeds	Subscribed feeds rendered on the home page, with visibility rules per entity, profile, group or user.
Reservations	Items marked reservable are booked on a calendar, with recurrence, per-user history, notifications and massive actions [11.0] .
Reports	Eight built-in reports: Default report, By contract, By year, Hardware financial and administrative information, Other financial and administrative information (licences, cartridges, consumables), Network report, Loan, and Status. Plugins can register more.
Saved searches	Any search can be saved, marked as a personal or global default, pinned to the home page, and attached to an alert that emails the result set on a schedule (SavedSearch_Alert, cronSavedSearchesAlerts).

Module	Detail
Impact analysis	An interactive dependency graph (Cytoscape) showing what an asset or service depends on and what depends on it, with compound grouping nodes, per-context styling and CSV export.

5.7 Administration

Module	Detail
Users	Local and directory-sourced accounts with multiple email addresses, categories, titles, per-entity and per-profile authorisations, and recursive visibility. LDAP import and synchronisation, single or bulk, plus a CLI sync command. Password policy with history and expiry, self-service reset by email, and a massive action to send reset mail [11.0] . A massive action re-applies authorisation rules [11.0] . Impersonation of lower-privileged users, gated by its own right [11.0] , restoring the original context on exit. TOTP two-factor enrolment [11.0] . Personal API tokens.
Groups	Hierarchical groups with members, manager and delegation flags; usable as ticket actors, asset owners, notification targets and approval bodies.
Entities	The multi-tenancy backbone. A tree of entities where almost every setting inherits from the parent and can be overridden locally — ticket defaults, notification options, survey configuration, autoclose delays, helpdesk tiles, address and administrative data. <i>Transfer</i> moves items between entities with per-relation rules, including the corrected handling of unused software [11.0] .
Profiles and rights	Granular rights per object type, expressed as bit masks over read, create, update, purge and object-specific verbs (assign, steal, own, change priority, read group, read assigned, survey, and more). A profile selects the standard or the simplified interface. Profiles can be cloned [11.0] .
Rules engine	34 rule collections covering ticket, change and problem business rules, asset rules, twenty-plus dictionaries, LDAP authorisation rules (<i>RuleRight</i> , including a <i>Deny login</i> action [11.0]), location assignment, entity and asset import, itemtype resolution, mail collector routing and software categorisation. Every collection supports ordering, testing, replay, and backup or restore.
Notification queue	<i>QueuedNotification</i> holds outbound mail for asynchronous dispatch, with retry, cleanup of sent items, and a stale-Ajax cleaner. <i>QueuedWebhook</i> does the same for webhooks.
Log viewer	Reads and filters the files under <code>files/_log</code> from the UI [11.0] .
Inventory	Native inventory configuration — see 5.4.2.
Forms	Administration of the form builder — see 5.3.2.

5.8 Setup

Section	Detail
General	Default values for new records, list and dropdown limits, session and cookie behaviour, password security policy, timezone, date and number formats, the URL base, and the maintenance-mode switch.
Appearance	Colour palettes, custom themes loaded from <code>files/_themes</code> [11.0] , layout options, rich-text editor behaviour, and configurable toast notification position [11.0] .
Dropdowns	Every reference table in the product, each with its own management screen and per-language translations: locations, states, ITIL categories, task categories, request sources, solution types, pending reasons, business criticality, document categories, user titles and categories, calendars and holidays, project states and types, planning event categories, network and OS reference data, and many more.
Components	The device catalogue, now grouped by category in the same way as dropdowns [11.0] .
Notifications	Global enable, email settings including SMTP with OAuth2, and templates with Twig-based content and a documented tag vocabulary, translated per language. Per-event recipient lists across 24 notification target classes, with recipient exclusions [11.0] . Browser (Ajax) notifications alongside email.
Webhooks	Outbound HTTP notifications with categories, payload templates, filtering and a delivery queue.
Service levels	SLM containers holding SLAs and OLAs with their escalation levels, criteria and actions — see 5.2.3.
Automatic actions	Around 48 scheduled tasks, each individually configurable: run mode (GLPI internal or external CLI), frequency, execution window, log level and retention. Per-task run logs. Listed in Appendix B.
Authentication	Local database, LDAP or Active Directory servers with replicates and a selectable TLS version [11.0] , IMAP/POP mail servers, CAS, X.509 certificates, SSO header variables, and OAuth2 identity providers (Google, Azure). Two-factor authentication policy, and a CLI command to clear a locked-out user's TOTP.
OAuth clients	Registered client credentials for API access, with scopes and grant types.
Receivers	Mail collectors that create and update tickets from a mailbox, driven by <code>RuleMailCollector</code> (with criteria renamed to <i>From email address</i> and <i>To email address</i> [11.0]), a content blacklist, tracking of not-imported messages, and collision handling.
External links	Per-itemtype links whose URL and file content are now Twig templates rather than a custom tag syntax [11.0] , with the itemtype association moved into the main form [11.0] .
Field unicity	Per-itemtype uniqueness constraints with configurable action and notification, plus field blacklists.
Asset definitions	Custom asset type designer — see 5.4.
Plugins and Marketplace	Plugin lifecycle from the UI or CLI, and a marketplace browser with an improved offline mode [11.0] , availability configuration [11.0] , and version-specific downloads from the CLI [11.0] .

6. Cross-cutting platform capabilities

These apply across every module above, and are the mechanisms most likely to be reused when implementing new requirements.

Capability	Detail
Search engine	Each itemtype declares search options; the engine composes them into criteria with AND, OR and NOT, nested groups, and meta-criteria that reach across related itemtypes (for example, tickets filtered by an attribute of the associated computer). Per-user column selection through <code>DisplayPreference</code> , plus sorting, pagination and a map view. A fuzzy matcher powers the quick global search.
Export	Any search result exports as CSV, PDF portrait, PDF landscape, ODS [11.0] , XLSX [11.0] or a names-only list.
Massive actions	A generic framework applying an action to every selected row of any list — update a field, add or remove a relation, assign, transfer between entities, delete, restore, purge. Actions are contributed per itemtype and, since 11.0, single-item actions are filtered by the item's own attributes.
Dashboards	A drag-and-resize grid editor (<code>GridStack</code>) with 19 widget types: pie, donut, half pie, half donut, bars, horizontal bars, stacked bars, horizontal stacked bars, multiple bars, multiple horizontal bars, lines, multiple lines, area, multiple areas, big number, multiple numbers, summary numbers, search result, editable markdown, and an article list. Dashboards are filterable, shareable, assignable as the default for a menu, seeded with demonstration data on a clean install [11.0] , and resettable to their as-shipped state [11.0] .
Notifications	Three channels — email, in-browser Ajax toast, and webhook — driven by a common event and target model, queued for asynchronous delivery, and templated in Twig with a documented tag vocabulary.
Automatic actions	The <code>CronTask</code> scheduler runs roughly 48 maintenance and business tasks, either inside GLPI on page hits or from an external scheduler calling <code>front/cron.php</code> . See Appendix B.
Audit trail	Field-level history on every item, an event log, the file log viewer, and a debug toolbar exposing SQL, timings and session data in development mode.
APIs	Three surfaces. (1) The legacy REST API, documented in <code>api.rest.md</code> . (2) The High-Level REST API, versions 2.0 to 2.3, with 21 resource controllers covering ITIL objects, assets, administration, management, projects, dropdowns, components, rules, notifications, knowledge base, reports and dashboards; RSQL filtering; and a generated OpenAPI document served through Swagger UI. Versions 2.0 to 2.2 are deprecated in favour of 2.3 [11.0.6] . (3) A GraphQL endpoint over the same schemas. Authentication by session token, user token or OAuth2. The XML-RPC API was removed in 11.0.
Command line	Around 70 bin/console commands: database install, update and schema integrity check; cache configuration and clearing; plugin install, activate and deactivate; marketplace search and download; plugin-to-core migrations (<code>FormCreator</code> , <code>Racks</code> , <code>Domains</code> , <code>Databases</code> , <code>Appliances</code> , <code>Generic object</code>); LDAP user sync; user create, enable, disable, delete, password reset and profile grant [11.0] ; security key rotation including OAuth keys [11.0.7] ; TOTP reset; maintenance mode; source and document integrity diagnostics; and requirement checks. Execution as root is blocked [11.0] .
Localisation	67 locales via gettext, right-to-left stylesheet generation, per-user language selection, translatable dropdown values, translatable knowledge base articles and reminders, and translatable forms and helpdesk strings.
Security	CSRF tokens, session hardening checks, configurable password policy with history and expiry, TOTP two-factor [11.0] , OAuth2 provider and client, Altcha CAPTCHA, an encrypted-secrets key (<code>GLPIKey</code>) with a rotation command, sodium-based cryptography, HTML sanitisation of rich text, SELinux and session configuration requirement checks, maintenance mode, and support for the well-known change-password URI [11.0] .
Extensibility	A plugin architecture with a broad hook catalogue, a marketplace client, custom asset definitions with custom fields, Twig content templates, and pluggable question types, dashboard widgets, notification targets, rule criteria and API schemas.

7. Quality assets already in the repository

Relevant to the next phase: there is an existing safety net to run before and after any customisation.

Asset	Detail
PHPUnit suites	621 test files under <code>tests/functional</code> and <code>tests/units</code> , with fixtures, a bootstrap, and a dedicated test configuration. Run through <code>phpunit.xml.dist</code> .
Cypress end-to-end	39 specs grouped by area: ITILObject, Kanban, Management, Planning, Setup, Tools, form, search, self-service, DebugMode, plus dropdown, entity, general configuration and maintenance specs. Launched with <code>npm run test:e2e</code> .
JavaScript tests	Jest, with a separate Vue project selector (<code>npm run test:vue</code>).
Playwright	A configuration is present at <code>playwright.config.ts</code> .
Static analysis	PHPStan with three baselines (general, missing iterable value types, specific), Psalm, Rector rule sets for core and for plugins, and PHP-CS-Fixer.
Linting	ESLint (flat config) and Stylelint.
LDAP fixtures	LDIF user data under <code>tests/LDAP/ldif</code> , loaded into the Compose OpenLDAP container; a console tool generates a matching LDAP object inside GLPI.
Mail fixtures	<code>tests/emails-tests</code> and <code>tests/imap</code> for exercising the mail collector.
Makefile	Wraps the common developer entry points, including running console commands inside the container.

8. Where to extend, when customisation starts

Offered as orientation for the next phase. The single most important architectural point: GLPI is designed to be extended through plugins and configuration. Editing core files under `src/` or `front/` works, but every future upgrade of the `11.0/bugfixes` branch then becomes a manual merge. Prefer the highest-level mechanism that satisfies the requirement.

If the requirement is...	Reach for...	Code touched
A new request or intake form, with its own fields and routing	The form builder. No code at all — sections, questions, conditions and a Ticket destination.	None
A new question type on forms	A class implementing <code>QuestionTypeInterface</code> , registered with <code>QuestionTypesManager</code> .	<code>src/Glpi/Form/QuestionType</code>
Different field behaviour on tickets (mandatory, hidden, prefilled, read-only)	Ticket templates. Configuration only.	None
Automatic classification, assignment or enrichment on ticket creation	Business rules for tickets — criteria and actions in the UI. Custom criteria or actions extend <code>RuleTicket</code> .	<code>src/RuleTicket.php</code>
A new scheduled behaviour	A <code>cronXxx</code> method plus a <code>CronTask</code> registration.	<code>src/CronTask.php</code> and the owning class
A new notification	A notification event on the relevant <code>NotificationTarget</code> subclass plus a template and tags.	<code>src/NotificationTarget*.php</code>
A new business object	A class extending <code>CommonDBTM</code> , search options, a <code>front/</code> page or a controller, Twig templates, and a schema migration.	<code>src/</code> , <code>front/</code> , <code>templates/</code> , <code>install/migrations/</code>
A new field on an existing object	A schema migration, a search option, and a Twig form override. Consider a plugin-owned side table to avoid touching core tables.	<code>install/migrations/</code> , <code>src/</code> , <code>templates/</code>
A new asset type with bespoke fields	Asset definitions with custom fields. No code.	None
A new API resource or endpoint	A High-Level API controller with its schema annotations; OpenAPI and GraphQL follow automatically.	<code>src/Glpi/Api/HL/Controller</code>
A new dashboard widget	A widget type plus a data provider.	<code>src/Glpi/Dashboard/Widget.php</code> , <code>Provider.php</code>
A new report or statistic	A <code>Stat</code> data class, or a saved search with an alert if a list will do.	<code>src/Glpi/Stat</code> , <code>src/Report.php</code>
A new CLI operation	A Symfony command extending <code>AbstractCommand</code> .	<code>src/Glpi/Console</code>
Anything that should survive upgrades cleanly	A plugin under <code>plugins/</code> , using the hook catalogue. This is the recommended default for bespoke work.	<code>plugins/</code>

One practical caution. The working tree already carries one local divergence from upstream (`.webpack.config.js`, section 4.2). Before customisation begins it is worth deciding how such divergences will be tracked — a dedicated branch, or a patch set — so that pulling upstream fixes on `11.0/bugfixes` stays a routine operation.

Appendix A — Key file references

Starting points for reading the code, verified in this checkout.

Concern	File	Note
Navigation and menus	src/Html.php:1250	<code>getMenuInfos()</code> — the authoritative module list
Shared ITIL behaviour	src/CommonITILObject.php	11,671 lines; statuses, actors, timeline, matrix
Tickets	src/Ticket.php	6,475 lines; types, rights, lifecycle
Changes	src/Change.php	Change record and workflow
Problems	src/Problem.php	Problem record
Approvals	src/CommonITILValidation.php	With <code>ValidationStep</code> and <code>ITIL_ValidationStep</code>
Service levels	src/LevelAgreement.php	Base for SLA and OLA
Ticket business rules	src/RuleTicket.php	Criteria and actions
Form builder	src/Glpi/Form/Form.php	Plus <code>QuestionType/</code> , <code>Destination/</code> , <code>Condition/</code>
Self-service home	src/Glpi/Helpdesk/Tile	Tile model and manager
Service catalogue	src/Glpi/Form/ServiceCatalog	Catalogue manager and providers
Search engine	src/Glpi/Search/SearchEngine.php	With <code>Output/</code> writers
Search output formats	src/Search.php:68-114	Output-mode constants
Dashboards	src/Glpi/Dashboard/Widget.php:113	<code>getAllTypes()</code> — the 19 widget types
Authentication	src/Auth.php:103-115	The eight authentication method constants
Two-factor	src/Glpi/Security/TOTPManager.php	TOTP enrolment and verification
High-Level API router	src/Glpi/Api/HL/Router.php	Version registration, v2.0 to v2.3
Legacy API docs	apirest.md	62,000 words of endpoint documentation
Custom asset types	src/Glpi/Asset/AssetDefinition.php	Plus <code>Capacity/</code> and <code>CustomFieldType/</code>
Native inventory	src/Glpi/Inventory/Inventory.php	Plus 33 handlers in <code>Asset/</code>
Reports	src/Report.php:70-112	The eight built-in report definitions
Statistics	src/Stat.php:1725-1765	The statistics menu definition
Automatic actions	src/CronTask.php	Scheduler; handlers live on the owning classes
Database schema	install/mysql/glpi-empty.sql	442 tables
Seed data	install/empty_data.php	Default profiles, entities, notifications, cron tasks
Requirements checks	src/Glpi/System/Requirement	What the installer verifies
Release notes	CHANGELOG.md	11.0.0 onwards; the source of the [11.0] markers
Dev stack	docker-compose.yaml	App on 8080, MariaDB, Mailpit on 8025, DBGate on 9000, OpenLDAP

Appendix B — Automatic actions present in the code

Cron handler methods discovered across the domain classes. Each corresponds to a CronTask entry that can be enabled, scheduled and logged independently from Setup. Names are the method identifiers; the UI shows translated labels.

Service desk and workflow

cronCloseTicket	cronSlaTicket	cronRecurrentItems
cronPurgeTicket	cronOlaTicket	cronWatcher
cronAlertNotClosed	cronApprovalReminder	cronPurgeDraftForms
cronCreateInquest	cronPendingreason_autobump_autosolve	cronPlanningRecall

Notifications and integrations

cronQueuedNotification	cronQueuedWebhook	cronMailgateError
cronQueuedNotificationClean	cronQueuedWebhookClean	cronSavedSearchesAlerts
cronQueuedNotificationCleanStaleAjax	cronMailgate	

Assets, inventory and stock

cronCartridge	cronCertificate	cronCleanoldagents
cronConsumable	cronContract	cronCleanorphans
cronSoftware	cronInfocom	cronCleanOrphans
cronCleanSoftware	cronDomainsAlert	
cronPurgeSoftware	cronReservation	

Platform maintenance

cronSession	cronCleantemp	cronCheckAllUpdates
cronCircularlogs	cronGraph	cronCheckDBreplicate
cronPurgeLogs	cronUnlockObject	cronTelemetry
cronLogs	cronPasswordExpiration	cronInfo
cronTemp	cronCheckUpdate	croncountAll

End of document. Compiled from read-only static analysis of the working tree at commit 76c38071d3 on branch 11.0/bugfixes, 11 August 2026. No application file was created, modified or deleted during the analysis. Feature statements describe code present in this checkout; they have not been confirmed against a running instance, because the application has not yet been installed on this machine (section 4).